

RepFC: Universal Structural Reparametrization Block for High Performance, Lightweight Deep Neural Networks

Shambhavi Balamuthu Sampath^{1,2†} Judeson Anthony Fernando^{1†} Moritz Thoma^{1,2} Nael Fasfous²
 Lukas Frickenstein² Pierpaolo Mori² Manoj Rohit Vemparala² Alexander Frickenstein²
 Ulf Schlichtmann¹ Walter Stechele¹

† Equal Contribution ¹Technical University of Munich ²BMW Group

Abstract

Structural reparameterization (SRP) employs a bipartite network design consisting of two stages: (1) an expanded multi-branch network during training, which is equivalent to (2) a collapsed lightweight network during inference. This approach improves the task metrics of the efficient inference-time network at the expense of longer training times for the expanded network. Recent works have adopted SRP for handcrafting efficient deep neural networks and have proposed universal multi-branch SRP blocks to expand only the convolutional (CONV) layers during training. However, these training schemes expand uniformly every CONV layer in a given deep neural network (DNN), which becomes impractical for modern large-scale DNNs. This work presents a simple and effective technique for structurally reparameterizing dense layers, unlocking additional potential for the technique. We achieve superior accuracy improvements with our proposed Reparametrizable Fully Connected (RepFC) block, by expanding only the dense layers of the classification head in SOTA DNNs, as opposed to expanding all CONV layers in related works. Additionally, we demonstrate how our RepFC block can be leveraged in transformers, where dense layers are employed throughout the network. Transformer architectures Swin-tiny, EfficientViT-B1 and FastViT-T8 show a 0.2, 0.3 and 0.75 p.p. improvement respectively, on the ImageNet-1K dataset. Our experiments demonstrate a 1.46, 0.85 and 1.14 p.p. improvements in AlexNet, MobileNet-V3-L and EfficientNet-B0 respectively on the ImageNet-1K dataset, with zero overhead in inference time. Code is available at <https://github.com/shamvbs/RepFC>.¹

1. Introduction

The last decade has seen tremendous contributions to deep learning through architectural improvements of deep neu-

ral networks (DNNs) aimed at boosting task metrics and network optimization techniques to realize efficient architectures on target hardware. Architectural improvements in neural networks often involve wider, deeper, and multi-branch design decisions, which typically increase network complexity and size. To manage these over-parameterized models, network optimization techniques such as quantization, pruning, and knowledge distillation are employed. Although these compression methods improve latency, they often lead to slightly degraded performance. This separation of the network design phase for improved task precision followed by a network compression phase to achieve real-time latency is not always ideal. Consequently, there is a growing interest in optimization techniques that can be integrated during the training phase.

Residual connections and multi-branch architectures have been superior to plain models for better representational capacity and for addressing vanishing gradients during training, thus boosting predictions [6]. The multiple paths capture diverse information from the feature maps to improve overall performance. However, such complex multi-branch networks are often memory inefficient and slow. The need for multiple parallel computations and extensive memory to store intermediate feature maps increases the overall computational load and latency during both training and inference. This significant memory overhead can limit the network's scalability and practicality in resource-constrained environments. To tackle this and get the best of both worlds, structural reparameterization (SRP) introduces a bipartite network design. During deployment, an efficient backbone is used, which during training has a multi-branch reparameterizable structure to enhance representational capacity and improve weight updates. Recent works have leveraged SRP techniques to design efficient architectures, allowing increased network depth and/or width during training without affecting inference speed [19, 22]. Although these studies focus on transformer network design, they apply SRP exclusively to the convolutional lay-

¹Correspondence to: shambhavi.balamuthu-sampath@tum.de

ers. Other research [4, 10] has standardized the dominant convolutional expansion approach with universally transferable SRP blocks. The growing popularity of SRP has highlighted a significant gap: its application to dense layers remains unexplored. Modern architectures often employ fully connected (FC) layers throughout the network, not just at the network head [13]. Therefore, the development of universally transferable SRP blocks for dense layers is highly motivated. Furthermore, this opens up new opportunities for additional performance improvements, complementing existing SOTA works in SRP. Our contributions are summarized as follows:

- To the best of our knowledge, we are the first to apply SRP to FC layers during training, enhancing model performance without altering the inference architecture, through *RepFC*, a universal multi-branch SRP block for dense layers.
- Inspired by SRP blocks in convolutional layers, we introduce two post-processing steps: train-time non-linear *batch normalization* (BN) layers and *learnable linear scaling factors* into RepFC branches. The latter ensures RepFC’s applicability to various architectures, including vision transformers, where BN may often conflict with the LayerNorm (LN) in its design.
- Experiments on ImageNet-1K demonstrate that our method achieves up to a 1.46 p.p. improvement in accuracy with a favorable trade-off in training time/batch. We demonstrate the scalability of our approach by applying RepFC to Swin-tiny with multiple dense layers, EfficientViT-B1 and FastViT-T8.
- We highlight the applicability of RepFC blocks for boosting pre-trained networks during their finetuning stage, with and without compressing them. Our experiments on ImageNet-1K consistently outperform vanilla finetuning by up to 1.33 p.p.

2. Related Works

The field of SRP for accuracy improvements has attracted significant attention in recent years. The expanded multi-branch network increases computational resource requirements and prolonged training times but modern DNN training with high-power GPUs can efficiently utilize these networks, enhancing performance without substantially increasing training costs. Nonetheless, it is crucial to find a balance between performance and efficiency to ensure practical training of larger and deeper models. We examine the SOTA approaches and categorize the related works into the following areas: (1) *train-time efficient* techniques that minimize training overhead while maintaining the diversity necessary to enhance capacity, (2) *universal SRP*

blocks that can be applied to any SOTA DNN, unlike hand-crafted architecture-specific SRP methods, and (3) *FC support* where SRP is applied to fully connected layers of the network.

Method	Conference	Train-time Efficient	Universal SRP block	FC Support
RepVGG [20]	CVPR 2021	-	✗	✗
RepViT [19]	CVPR 2024	-	✗	✗
RepNAS [21]	ICME 2023	✓	✗	✗
DBB [4]	CVPR 2021	✗	✓	✗
OREPA [10]	CVPR 2022	✓	✓	✗
Ours		✓	✓	✓

Table 1. **Comparison to SOTA.** Our work proposes universal SRP blocks for the currently unsupported FC layers in SOTA.

As seen in Table 1, the works that do not propose universal SRP blocks [19, 20, 22] are all architecture-specific and leverage SRP to boost their network competitiveness in terms of accuracy. The training time for these works depends on the specific architecture under design, making it difficult to compare directly with other universal multi-branch SRP (MB-SRP) blocks based approaches. Their method relies on batch normalization (BN) to merge multiple train-time SRP branches into a single inference-time branch, and none have explored the possibility of structurally reparametrizing the fully connected layers. In contrast, we aim to present a universally applicable SRP block that can be used in any DNN with dense layers. Another category of research, proposes universal SRP blocks [4, 10] with diverse branches and transforms, applicable to any convolutional neural network (CNN). However, these approaches also restrict their application to the convolutional layers. For instance, DBB [4] demonstrates accuracy improvements by applying the same SRP block to all convolutional layers of a network, albeit at the cost of significant training overhead, making it especially impractical for modern, large networks. To address this, [21] demonstrates that not all layers require the same SRP block and performs a search for the optimal DBB block on different layers, naming it ODBB. Another work, OREPA [10], takes a different approach by proposing an online SRP block that linearizes the block by moving the batch normalization layer outside the block. With a focus on efficient training, OREPA employs a two-stage pipeline: first, linearize the MB-SRP block with linear scaling factors, and then to reparametrize the MB-SRP block online during training. The idea is that efficient training time would enable the exploration of more complex MB-SRP blocks. However, we hypothesize that adding complexity to the SRP blocks creates an overhead in additional hyperparameter tuning to avoid overfitting issues. Therefore, with our RepFC block, we investigate two different variants for block complexity (three branches and five branches) and present the impact of including learnable scaling factor and BN in each RepFC branch.

3. Background and Method

In this section, we introduce our structurally reparametrizable fully connected (RepFC) block. First, we briefly explain how MB-SRP blocks are designed for convolutional (CONV) layers. Then, we analyze the linearity of an FC layer as preliminaries. Next, we present the design of RepFC and its variants, followed by the formal algorithmic definition of the proposed method.

3.1. MB-SRP blocks for CONV Layer

The structural reparametrization of a parallel multi-branch convolutional structure is straightforward due to the linearity of convolution, which allows us to merge multiple branches into one. Consider an example with three parallel convolutional layers: one with a 3×3 kernel, one with a 1×1 kernel, and an identity branch.

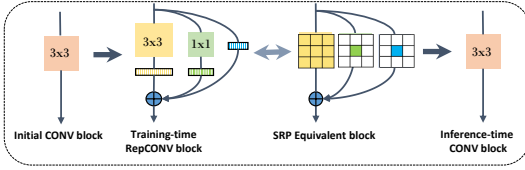


Figure 1. The perspective of the parallel CONV branches being merged after training. The identity branch is also a 1×1 conv with an identity matrix as the kernel. Figure adapted from [20].

According to the linearity of convolution, we can express the output Y as:

$$\begin{aligned} Y &= (W_{3 \times 3} * X + b_{3 \times 3}) + (W_{1 \times 1} * X + b_{1 \times 1}) + (I * X + b_I) \\ &= (W_{3 \times 3} + W_{1 \times 1} + I) * X + (b_{3 \times 3} + b_{1 \times 1} + b_I) \\ &= W_{merged} * X + b_{merged} \end{aligned} \quad (1)$$

where $W_{3 \times 3}$ and $W_{1 \times 1}$ are the weights of the 3×3 and 1×1 convolutional kernels, respectively, and I is the identity branch with weights set to 1. The terms $b_{3 \times 3}$, $b_{1 \times 1}$, and b_I represent the biases for the respective branches.

To effectively merge these branches, it is crucial to align the spatial centers of the different kernels through appropriate padding. When multiple two-dimensional kernels with compatible sizes operate on the same input with identical strides, producing outputs of the same resolution, their outputs can be summed. This summation allows for the combination of these kernels at corresponding positions to form an equivalent kernel that produces the same output [3]. Consequently, the unified result can be computed as a single convolution operation, applying the merged weights W_{merged} and the combined biases b_{merged} to the input X without loss of information.

3.2. The Linearity of FC Layer

Similar to the convolutional (CONV) operation, the linearity of the fully connected (FC) layer enables the merging of multiple parallel branches into a single branch. The FC layer processes a flattened input $X \in \mathbb{R}^{n \times H_{in}}$, where H_{in} denotes the number of input features and n represents the batch size. It produces a one-dimensional output vector $Y \in \mathbb{R}^{n \times H_{out}}$, where H_{out} signifies the number of output features. The operation of the FC layer is defined as:

$$Y = XW + b \quad (2)$$

where $W \in \mathbb{R}^{H_{in} \times H_{out}}$ is the weight matrix and $b \in \mathbb{R}^{H_{out}}$ is the bias vector. The linearity properties, *additivity* and *homogeneity*, allow us to efficiently combine multiple branches.

Additivity The output of the FC layer for the sum of two input matrices is equal to the sum of the outputs for each input matrix processed separately. For any two weight matrices $W_1, W_2 \in \mathbb{R}^{H_{in} \times H_{out}}$, additivity is expressed as:

$$X(W_1 + W_2) = XW_1 + XW_2 \quad (3)$$

Homogeneity The output of the FC layer for a scaled input matrix is equal to the scaled output of the original input matrix. For a scalar $\alpha \in \mathbb{R}$ and a weight matrix $W \in \mathbb{R}^{H_{in} \times H_{out}}$, homogeneity is expressed as:

$$X(\alpha W) = \alpha(XW) \quad (4)$$

These properties enable the design of our RepFC MB-SRP block to handle different branches and their respective transformations of the input, ultimately resulting in a unified, single FC operation with combined weights and biases.

3.3. Design of RepFC Block

The core idea behind our RepFC block is to exploit the variable contribution of different components in the feature vector that is fed to the fully connected (FC) layers. In a single FC layer, all feature vector values are connected to all weights, implying that each value contributes equally to the final output. However, with RepFC, we explore our hypothesis that certain feature vector components, i.e. certain subsets of the feature vector, are sometimes more significant than others for the model's predictions. By emphasizing these components, the network learns to better separate classes during training. This is in line with previous studies that emphasize the regions of an input image most pertinent to the classification task [4, 10]. On that note, the key design aspects of our RepFC block can be listed as follows:

Exploiting Variable Feature Contribution Unlike previous works that fed identical inputs to each expanded SRP branch, the RepFC block takes a different approach. It feeds different feature subsets (masked inputs) to different FC branches, allowing it to exploit the variable contribution of different feature vector components.

Ensemble of FC layers with masked inputs The RepFC block can be perceived as an ensemble of FC layers with complementary masking patterns applied to the inputs. This aims to boost the robustness and accuracy of predictions by leveraging the expanded expressiveness of the network.

Equal treatment for feature subsets To ensure each feature vector value is treated equivalently during training, the RepFC block is systematically designed with complementary pairs of branches. In each pair, one branch processes half the feature vector, while the other branch processes the other half.

Adaptability to dataset difficulty The RepFC block is designed with two variants, RepFC-III and RepFC-V, which differ in the number of pairs of branches. Depending on the dataset difficulty (e.g., CIFAR-100 vs ImageNet-1K), the appropriate RepFC variant can be selected for a good trade-off between over-parameterization and training time.

3.4. RepFC Variants

Based on the above design principles, two RepFC variants are proposed (shown in Fig. 2) and detailed as follows: **RepFC-III**: In the first FC branch of this block, the first half of the feature vector is masked, while the second half is retained. In the complementary branch, the second half of the feature vector is masked, and the first half is retained. **RepFC-V**: This is an extension of RepFC-III with two additional branches. One FC branch of this pair has all even-indexed values of the vector masked, while the other has all odd-indexed values masked, retaining the remaining values. For a memory-efficient implementation (i.e., reduced parameters during training), we slice and retain only the relevant portions of the masked inputs in the branches.

3.5. Lossless RepFC Merging

As a study case, we present the training-time implementation of RepFC-III with BN, that enables a lossless merging of the block during inference. Given a flattened input feature map $X \in \mathbb{R}^{H_{in}}$, we can express the output Y as follows:

$$Y = BN_1(W_1 * X) + BN_2(W_2 * X_1) + BN_3(W_3 * X_2) \quad (5)$$

where $X_1, X_2 \in \mathbb{R}^{H_{in}/2}$ are complementary subsets such that together they constitute the entire input feature X , i.e., $\text{cat}(X_1, X_2) = X$, here the function $\text{cat}(\cdot)$ denotes the concatenation operation. In our example, $X_1 = X[: H_{in}/2]$

and $X_2 = X[H_{in}/2 :]$ (see Fig. 2). Thanks to the linearity properties of FC layers, they can be merged with its subsequent one-dimensional BN layer, similar to RepCONV+BN merge [4, 10]. Let us denote the resulting new second-order tensor W' and bias b' for a given FC layer after the one-dimensional BN folding process.

$$\begin{aligned} Y &= (W'_1 * X + b'_1) + (W'_2 * X_1 + b'_2) + (W'_3 * X_2 + b'_3) \\ &= (W'_1 * X + b'_1) + (\text{cat}(W'_2, W'_3) * \text{cat}(X_1, X_2) + b'_2 + b'_3) \end{aligned} \quad (6)$$

The concatenation operation is feasible and constitutes a lossless transformation, as the complementary subset branches are specifically designed to avoid overlapping inputs. Consequently, we obtain a single merged fully connected layer, denoted as $\text{FC}_{\text{merged}}$.

$$\begin{aligned} Y &= (W'_1 + \text{cat}(W'_2, W'_3)) * X + (b'_1 + b'_2 + b'_3) \\ &= W'_{\text{merged}} * X + b'_{\text{merged}} \\ &= \text{FC}_{\text{merged}}(X) \end{aligned} \quad (7)$$

3.6. Post-Processing of FC Outputs

In addition to the branch complexity variants, our RepFC block also investigates two options for post-processing the FC outputs: (1) **Learnable Linear Scaling Factors**: As first proposed in [10] for the CONV layer SRP, we employ learnable linear scaling factors for the outputs of each FC branch. This allows the model to learn the appropriate scaling for each feature subset, further enhancing the exploitation of variable feature contributions. (2) **Batch Normalization (BN)** As seen in [4], we enhance the performance of the model by applying BN to each FC branch before concatenating their outputs. By normalizing each branch's output separately, the concatenated features are well-conditioned, each feature subset contributes effectively without being dominated by any branch, all while lowering the risk of training instabilities. Here, we disable the biases of the FC layers to avoid redundancy brought by the BN layer's bias.

3.7. RepFC Pipeline

Figure 2 illustrates a sample initial network comprising a backbone and a head going through the RepFC pipeline with head expansion blocks. The head expansion step for a training-time network and a head squeezing step to obtain an efficient inference-time network is shown. Our lossless SRP method specifically targets the expansion of fully connected (FC) layers, which are typically located in the classification head of models. The expansion can be configured to use either a RepFC-III or RepFC-V block.

Algorithm 1 provides a detailed, step-by-step procedure for implementing our RepFC method. The algorithm starts by initializing the necessary parameters and layers based on the chosen RepFC variant. For RepFC-III, three branches are created, while for RepFC-V, five branches are created.

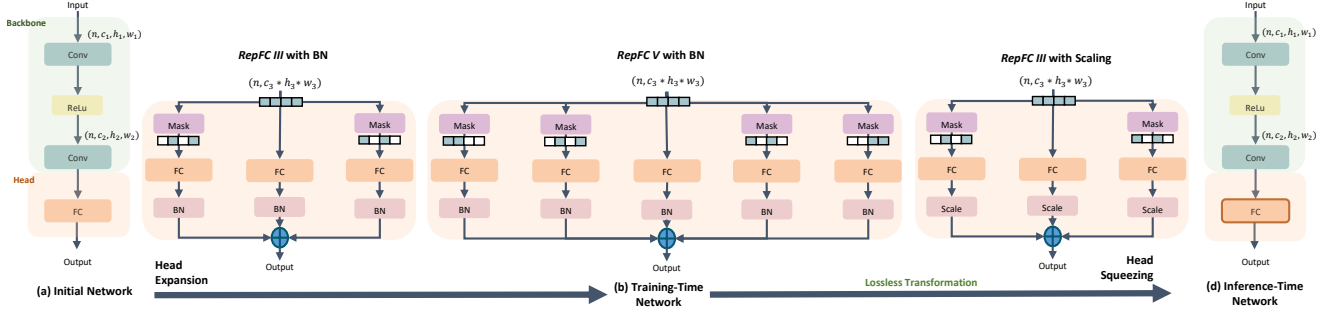


Figure 2. **High level overview of the RepFC pipeline.** The FC layer of the initial network is subjected to training-time expansion with any of the RepFC variants shown (RepFC-III, RepFC-V). Moreover, either the learnable linear scaling factor or BN layer can be employed in the RepFC branches. Post training, the expanded blocks is collapsed into the original layer for efficient inference.

Each branch processes a different subset of the input feature vector, either retaining the original input, slicing half of the input values, or alternating slicing even and odd indices. The outputs from each branch are then scaled with a learnable scaling factor for each branch, encompassed in the vector α , or normalized with BN layers before being summed to produce the final output Y , which is then returned by the algorithm. The difference between our RepFC block variants lies in the number of branches and the specific subset of input features provided to each branch, as presented in Fig. 2. This architectural choice allows the multi-branch structure to optimize in diverse directions based on the input features being processed. Notably, the expanded training-time model adheres to the same training recipe as the initial network, thereby isolating the benefits of the SRP method.

After the training phase, the expanded FC layer is collapsed back into a single FC layer. First, the linear scaling factor or BN layer is fused with its corresponding FC layer, similar to the approach used for convolutional layers in the DBB [4] work. Then, all FC layers are merged by addition into a single layer. This entire post-training process of collapsing into a single FC layer is a lossless transformation, making the method both simple and attractive.

4. Evaluation

We conduct experiments on CIFAR-100 [11] and ImageNet-1K [2] datasets using a range of SOTA architectures to evaluate the accuracy improvements obtained with RepFC blocks. The experimental configurations for CIFAR-100 are standardized across all networks used, incorporating standard data augmentation techniques including padding the input images to 40×40 , random cropping, and random horizontal flipping [6]. We employed a cosine learning rate decay scheduler over a total of 300 epochs, with an initial learning rate of 0.1 and a 10-epoch warm-up. All experiments utilized a weight decay of 1×10^{-4} . Experiments with ResNet18 and SENet18 were conducted with a batch size of 256 on two NVIDIA

Algorithm 1: RepFC

```

1: Parameters:  $in_{features}=d, out_{features}, use_{scaling}, use_{bn}$ 
2: Initialization:
3:  $nb_{branch} \leftarrow 3$  if RepFC-III, else  $nb_{branch} \leftarrow 5$  if RepFC-V
4: Initialize original linear layer  $f_{c0}$  with shape
   ( $out_{features}, in_{features}$ )
5: Initialize  $(nb_{branch} - 1)$  linear layers  $f_{ci}$  with shape
   ( $out_{features}, in_{features}/2$ )
6: if  $use_{scaling}$  then
7:   Initialize vector  $\alpha$  of size  $nb_{branch}$ 
8: else if  $use_{bn}$  then
9:   Initialize BN layers  $bn_i$  with shape  $out_{features}$ 
10: end if
11: Forward Pass:
12: Create inputs:  $X_1, X_2, X_3 \leftarrow \{x, x[:, : d/2], x[:, d/2 :]\}$ 
13: Add inputs:  $X_4, X_5 \leftarrow \{x[:, : 2], x[:, 1 : 2]\}$  if RepFC-V
14: Process each input through its respective  $f_c$  branch
15: and apply scaling or BN:
16: if  $use_{scaling}$  then
17:    $Y \leftarrow \sum_{i=1}^{nb_{branch}} \alpha[i] \cdot f_{ci}(X_i)$ 
18: else if  $use_{bn}$  then
19:    $Y \leftarrow \sum_{i=1}^{nb_{branch}} bn_i(f_{ci}(X_i))$ 
20: end if
21: return  $Y$ 

```

V100 GPUs (32 GB memory each). Experiments with other architectures were conducted with a batch size of 128 on a single NVIDIA V100 GPU (32 GB memory). Additionally, the experimental settings for ImageNet-1K are architecture-specific and are summarized in Table 3. Throughout this study, the same seed (either 5 or 42) was used for all experiments for a given architecture.

4.1. Accuracy Improvements

The set of architectures benchmarked is presented in Table 2. Based on our analysis of different RepFC variants and their individual post-processing steps, the best results were achieved with RepFC-III followed by BN on the CIFAR-100 dataset, and RepFC-V followed by BN on

the ImageNet-1K dataset, to balance over-parametrization and dataset difficulty. For transformers, the application of BN becomes incompatible and can deteriorate performance when architectures utilize Layer Normalization (LN). Consequently, it is necessary to employ learnable scaling factors for post-processing FC outputs. In Swin-tiny, with 64 FC layers, RepFC-III with linear scaling was applied to all six MLP blocks and the head. Conversely, EfficientViT-B1 and FastViT-T8 have only two FC layers in the head. EfficientViT-B1, which uses LN, employs linear scaling for post-processing and FastViT-T8, which uses BN, is compatible with our RepFC BN variant. As seen in Table 2, all architectures demonstrated positive improvements in classification accuracy. On the CIFAR-100 dataset, one of the most substantial gains were observed in EfficientNet-B0, which saw an increase of +0.58 p.p., illustrating that even performance-optimized architectures can benefit from our novel RepFC blocks. Moderate complexity models such as ResNet-18 and SENet-18 also showed consistent improvements on CIFAR-100, highlighting the versatility of RepFC blocks in enhancing performance across different levels of architectural complexity. On the ImageNet-1K dataset, AlexNet exhibited the most improvement with a +1.46 p.p. boost, highlighting the potential for simpler architectures relying on FC layers to achieve significant gains. Following this is EfficientNet-B0, that showed a substantial performance enhancement on ImageNet-1K +1.14 p.p., indicating that the RepFC block can drive noticeable accuracy gains even in more complex, performance-optimized architectures. While lightweight models such as MobileNet-V3-S and transformer-based architectures like Swin-tiny and EfficientViT-B1 exhibited modest improvements, the overall trend confirms the broad applicability and effectiveness of RepFC blocks in enhancing neural network performance across a diverse array of model designs and datasets, including transformers. This is once again seen in FastViT-T8, with a remarkable 0.75 p.p. accuracy improvement on ImageNet-1K.

4.2. SOTA Comparison

We get on a head-to-head comparison to the existing SOTA works providing universal SRP blocks for CONV layers. DBB [4], in its work has benchmarked its CONV MB-SRP block on a range of SOTA models while OREPA [10] limited its evaluation to ResNets and RepVGG [20]. As presented in Table 4, we evaluate our RepFC block along with DBB and OREPA on the architectures - AlexNet, ResNet-18, and MobileNet. With this, we provide a perspective of training time/batch saved with RepFC blocks that can boost performance even when employed alone. Moreover, we emphasize that RepFC is orthogonally beneficial to the existing CONV MB-SRP blocks. As for the experimental settings, AlexNet** and ResNet-18 follow the same configurations

Dataset/ Variant	Architecture	Top-1 Acc (%)		Accuracy ↑ [p.p.]
		Baseline	RepFC	
CIFAR-100 RepFC-III	AlexNet[12]	59.69±0.22	60.31±0.24	+0.62
	ResNet-18[6]	78.77±0.23	79.28±0.03	+0.51
	SENet-18[9]	78.48±0.17	78.90±0.12	+0.42
	MobileNet-V2[14]	75.21±0.01	75.75±0.26	+0.54
	EfficientNet-B0[17]	72.60±0.28	73.18±0.38	+0.58
ImageNet-1K RepFC-V	AlexNet[12]	59.95	61.41	+1.46
	VGG-16 [4]	72.85	73.58	+0.73
	ResNet-18[6]	70.91	71.41	+0.50
	SENet-18[9]	71.80	71.94	+0.14
	MobileNet-V3-S[7]	66.93	67.28	+0.35
	MobileNet-V3-L[7]	73.93	74.78	+0.85
	EfficientNet-B0[17]	75.44	76.58	+1.14
	Swin-Tiny[13]	81.35	81.55	+0.20
	EfficientViT-B1[1]	78.09	78.40	+0.31
	FastViT-T8[18]	75.72	76.47	+0.75

Table 2. **Top-1 Accuracy on CIFAR-100 and ImageNet-1K models.** Accuracy improvement of baseline models with the RepFC-III & RepFC-V blocks. CIFAR-100 results are an average of 3 runs.

Architecture	GPU NVIDIA	#GPUs	Epochs	Batch size	Init learn rate	Weight decay
<i>CNN Architecture</i>						
AlexNet[12]	V100	4	120	1024	1×10^{-1}	1×10^{-4}
VGG-16[15]	A100	2	120	256	1×10^{-1}	1×10^{-4}
ResNet-18[6] / SENet-18[9]	V100	4	120	256	1×10^{-1}	1×10^{-4}
MobileNet-V3-S/L[7]	H100	2	300	2048	4×10^{-3}	5×10^{-2}
EfficientNet-B0[17]	A100	2	150	256	5×10^{-2}	4×10^{-5}
<i>CNN-Transformer Hybrid Architecture</i>						
Swin-tiny[13]	H100	2	320	1024	1×10^{-3}	5×10^{-2}
EfficientViT-B1[1]	A100	4	320	1024	4×10^{-3}	1×10^{-1}
FastViT-T8[18]	H100	4	300	1024	1×10^{-3}	1×10^{-1}

Table 3. **Training Configurations** for ImageNet-1K models.

provided in Table 3 and for MobileNet we used the configurations provided in DBB [4] that consisted of shorter training for 90 epochs and standard augmentation techniques for ImageNet-1K dataset. The chosen RepFC variant is RepFC-V with BN that brought superior performance for ImageNet-1K. As presented in Table 2, in AlexNet**, with its three FC layers forming 95% of the model parameters, RepFC achieves a +1.46 p.p. improvement over the baseline and outperforms DBB by +0.43 p.p. When combined with DBB, the accuracy reaches 62.21%, the highest among all configurations (+2.26 p.p.), indicating the complementary effects of RepFC and DBB, with a small 13.6% overhead in training time. In the case of ResNet-18, RepFC improves accuracy by 0.5 p.p. over the baseline with a minimal 2.4% training time overhead. While DBB provides a slightly higher accuracy improvement due to the dominant CONV layers, it comes with a substantial 237.8% training time overhead. This was addressed by the OREPA block that outperforms DBB by 0.2 p.p. with a comparatively lower training time overhead of 104.8%. Combining RepFC with OREPA results in the highest accuracy of 72.27% with

Architecture	Reparam	Top1-Acc (%)	Training Time /Batch (ms)	Training Time Overhead (%)	
				Over Baseline	Brought by RepFC
AlexNet**[4]	None = Baseline	59.95	250	-	-
	RepFC	61.41 (+1.46)	277	+10.8	+10.8
	DBB	60.96 (+1.01)	273	+9.2	-
	DBB + RepFC	62.21 (+2.26)	284	+13.6	+4.0
ResNet-18[6]	None = Baseline	70.91	82	-	-
	RepFC	71.41 (+0.50)	84	+2.4	+2.4
	DBB	71.76 (+0.85)	277	+237.8	-
	DBB + RepFC	72.05 (+1.14)	280	+241.5	+1.1
	OREPA	71.96 (+1.05)	168	+104.9	-
MobileNet[8]	OREPA + RepFC	72.27 (+1.36)	173	+111.0	+3.0
	None = Baseline	72.23	87	-	-
	RepFC	72.48 (+0.25)	89	+2.3	+2.3
	DBB	72.57 (+0.34)	417	+379.3	-
	DBB + RepFC	72.91 (+0.68)	425	+388.5	+1.9
	OREPA	72.33 (+0.10)	212	+143.7	-
	OREPA + RepFC	72.41 (+0.18)	220	+152.9	+3.8

Table 4. **SOTA Comparison for Top-1 Accuracy on ImageNet-1K dataset.** We incrementally add the SOTA CONV SRP blocks DBB [4] and OREPA [10], our RepFC-V block, and a combination of both CONV and FC SRP blocks. AlexNet** depicts the modified AlexNet with BN incorporated after each CONV layer as presented in [4]. The training time overhead is architecture-specific, i.e., in large networks with dominant CONVs, RepFC still boosts performance without the overhead of training time as in DBB or OREPA.

a 111.0% training time overhead, outperforming the standalone SOTA OREPA block by 0.3 p.p.. For MobileNet, RepFC improves accuracy by 0.25 p.p. over the baseline with a negligible 2.3% training time overhead. Notably, this is outperforming OREPA as well as combining RepFC with OREPA. This observation therefore validates our hypothesis that OREPA blocks only work well on architectures with larger kernels (7x7 as in ResNets) and are not directly transferable to other architectures, including AlexNet. Furthermore, the last FC layer in MobileNet contributes to 24.22% of the model parameters where RepFC can be leveraged effectively. These results highlight that the RepFC block offers competitive or superior accuracy improvements with a minimal computational overhead, and the flexibility to combine it with other CONV SRP blocks like DBB or OREPA yielding the highest accuracy improvements overall. The trade-off between accuracy and training time is architecture specific, especially in architectures with dominant FC layers, RepFC makes an attractive choice for efficient model enhancement.

4.3. Ablation Study

We conduct an ablation study in Table 5 to investigate the individual contributions of both RepFC variants, examining the need for branch complexity on various SOTA models. For a fair comparison, we also consider the baseline with duplicating the original FC layer without any masking. Our results show that simply introducing over-parameterization during training does not consistently improve accuracy; in fact, adding an extra FC layer can degrade performance. Additionally, to justify our choice of having complementary

branch pairs during the training process, we conduct experiments with dropout [16] instead of a fixed mask. The results highlight that the networks performed very poorly, with even a drop of 1.12 p.p. for EfficientNet-B0. Regarding RepFC-V, it improves predictions of only EfficientNet-B0 with 0.51 p.p. In contrast, the RepFC-III block consistently improves the FC layer’s ability to extract relevant features, achieving peak accuracies of 79.28%, 78.90% and 73.19% for ResNet-18, SENet-18 and EfficientNet-B0, respectively. As a conclusion, for simpler datasets like CIFAR-100, having more than two extra exploration branches is less beneficial, while it is more useful for complex datasets like ImageNet-1K, Table 2.

Architecture	Top-1 Acc (%)		
	ResNet-18[6]	SENet-18[9]	EfficientNet-B0[17]
Baseline	78.77±0.23	78.48±0.17	72.60±0.29
+ Extra FC layer	78.72±0.27 (-0.05)	78.21±0.29 (-0.27)	72.61±0.38 (+0.01)
+ MB-III Dropout	77.94±0.14 (-0.83)	78.46±0.20 (-0.02)	71.48±0.04 (-1.12)
+ RepFC-V	78.63±0.25 (-0.14)	78.41±0.17 (-0.07)	73.11±0.24 (+0.51)
+ RepFC-III	79.28 ±0.03 (+0.51)	78.90 ±0.12 (+0.42)	73.19 ±0.38 (+0.59)

Table 5. **An ablation study on CIFAR-100 dataset.** This table compares the performance of different variants of RepFC and examines the impact of adding an extra FC layer and using dropout in RepFC-III. The results represent the average of 3 runs, with experiments conducted using fixed seeds {5, 42, 93}.

4.4. Finetuning with RepFC

In this section, we prove the usefulness of RepFC blocks when you do not need to train your architecture from scratch and can start from a pre-trained checkpoint.

Architecture	#Params (M)	Top-1 Acc (%)			Finetuning Epochs
		Baseline	Vanilla Finetuning	RepFC-V Finetuning	
AlexNet[12]	61.10	59.95	60.53 (+0.58)	61.94 (+1.99)	100
AlexNet _{1.27x}	48.83	17.31 (-42.64)	59.81 (-0.14)	61.14 (+1.19)	120
ResNet-18[6]	11.69	70.91	70.94 (+0.03)	71.74 (+0.83)	100
ResNet-18 _{1.28x}	09.23	08.31 (-62.60)	70.04 (-0.87)	71.20 (+0.29)	120
ResNet-50 _{1.28x}	20.42	00.63 (-79.41)	79.78 (-0.26)	79.98 (-0.06)	300
SENet-18[9]	11.78	71.80	71.81 (+0.01)	72.08 (+0.28)	100
SENet-18 _{1.28x}	09.29	09.66 (-62.14)	71.04 (-0.76)	71.28 (-0.52)	120
MobileNet-V3 L[7]	05.18	73.93	74.10 (+0.17)	75.03 (+1.10)	225
MobileNet-V3 L _{1.39x}	04.26	00.10 (-73.83)	73.22 (-0.71)	73.75 (-0.18)	300
EfficientNet-B0[17]	05.29	75.44	75.93 (+0.49)	76.72 (+1.28)	225
EfficientNet-B0 _{1.28x}	04.31	00.98 (-74.46)	74.97 (-0.47)	75.94 (+0.50)	300

Table 6. **Finetuning pre-trained networks with and without RepFC-V on ImageNet-1K dataset.** Networks with a subscript ($1.2x \times$) denote pruned networks and their speedup. In these pruned cases, finetuning is done for longer epochs to recover the lost accuracy. In all cases RepFC blocks help enhance DNNs outperforming vanilla finetuning.

We show two possible scenarios using the ImageNet dataset, (i) finetuning the architecture after restoring the pre-trained checkpoint and (ii) finetuning the architecture after carrying out channel pruning causing a $1.2x \times$ speedup in the architecture. For pruning, we used GroupNorm

Importance-based channel pruning from [5] with a pruning ratio of 0.1. The reduction in total parameters is presented in table 6 with the speedup mentioned as subscript to the network names. We carry out retraining with and without the RepFC-V BN block. Clearly, RepFC-V finetuning consistently achieves higher accuracy than both the baseline and vanilla finetuning methods. Notably, AlexNet achieves an accuracy improvement from 59.95% to 61.94%, and EfficientNet-B0 improves from 75.44% to 76.72%.

4.5. Pruning the last FC layer

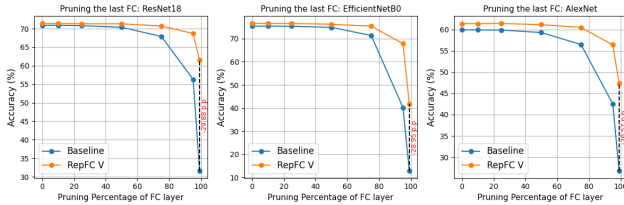


Figure 3. The impact of pruning the last sensitive layer responsible for classification on the ImageNet-1K dataset.

Using the RepFC block during training helps highlight the most important input features in each branch. This leads to the merging process assigning more weight only to the features that matter the most. Leveraging this observation, we prune the last FC layer, which is conventionally not pruned in literature as it is considered sensitive and responsible for the final classification. As a pruning approach, we use an unstructured magnitude-based method, progressively pruning the last FC layer from 10% to 99% of the weights with the lowest absolute magnitude. As shown in Fig. 3, even though the accuracy drops as we prune further, the drop is less severe when using the RepFC block compared to the baseline. Notably, there is on an average 26.5% accuracy gap between the architecture trained with RepFC and the one without, when pruning 99% of the weights. Specifically, for the previously mentioned pruning ratio, ResNet-18 achieves 61.49% accuracy, which is less than a 10 p.p. drop. These results demonstrate the effectiveness of the RepFC block in preserving important input features even under extreme pruning conditions, making it a promising approach for efficient model deployment.

4.6. Online Reparametrization of RepFC

To further streamline the training time of these MB-SRP RepFC blocks, we propose online reparametrization of dense layers comparable to the approach OREPA suggested for CONV layers [10]. As shown in algorithm 2, Online RepFC dynamically computes the final weights during the forward pass, thus eliminating the need for multiple forward passes through separate branches. This approach reduces computational overhead and simplifies the weight combination step, leading to faster training times. Additionally,

by applying a single linear transformation and BN after the weight computation, Online RepFC offers a more efficient and scalable solution, making it suitable for a wide range of architectures. The savings in training time per batch with this method have been presented in Table 7. In networks where FC makes less than 5% of the total parameters, the online mode boosted performance over RepFC-V BN.

Architecture	Top-1 Acc (%)			Training-Time/Batch (ms)			Baseline FC Params (%)
	Baseline	RepFC-V	Online RepFC-V	Baseline	RepFC-V	Online RepFC-V	
AlexNet[12]	59.95	61.41 (+1.46)	61.23 (+1.28)	250	277	266	95.95
ResNet-18[6]	70.91	71.41 (+0.50)	71.76 (+0.85)	82	84	83	04.39
SENet-18[9]	71.80	71.94 (+0.14)	72.24 (+0.44)	88	91	87	04.35
MobileNet-V3 L[7]	73.93	74.78 (+0.68)	74.07 (+0.14)	532	575	535	48.45
EfficientNet-B0[17]	75.44	76.58 (+1.14)	76.54 (+1.10)	168	175	170	24.22

Table 7. Comparison of Online RepFC with Baseline and RepFC-V in terms of Top-1 Accuracy and Training Time per Batch (ms) for ImageNet-1K dataset. The last column shows the percentage of parameters from FC layers over the total number of parameters for each architecture.

Algorithm 2: Online RepFC

- 1: **Parameters:** $in_{features}$, $out_{features}$, use_{bn}
- 2: **Initialization:**
- 3: $nb_{branch} \leftarrow 3$ if RepFC-III, else $nb_{branch} \leftarrow 5$ if RepFC-V
- 4: Initialize weight tensor w_0 with shape $(out_{features}, in_{features})$
- 5: for original linear layer
- 6: Initialize $(nb_{branch} - 1)$ weight tensors w_i with shape $(out_{features}, in_{features} // 2)$ for the extra branches
- 7: Initialize nb_{branch} vectors $vect_i$ with shape $out_{features}$
- 8: Initialize one BN layer bn with shape $out_{features}$ if use_{bn}
- 9: **Forward Pass:**
- 10: $weight \leftarrow \sum_{i=1}^{nb_{branch}} w_i \times vect_i$
- 11: $Y \leftarrow F.linear(x, weight)$
- 12: $Y \leftarrow bn(Y)$ if use_{bn}
- 13: **return** Y

5. Conclusion

This paper introduces RepFC, a universal multi-branch structurally reparametrizable block that can be leveraged on SOTA deep neural networks with dense layers. Our experiments demonstrate that RepFC achieves a favorable trade-off between training time and performance boost with no overhead in inference complexity/latency. RepFC closes the gap in the existing works, where SRP methods are predominantly applied to the convolutional layers only. With modern DNNs and transformers increasingly adopting dense layers throughout the network, we believe our work is a valuable addition to the field. Future work involves exploring further post-processing patterns of FC multi-branch outputs in various network designs to maximize the performance boost.

References

- [1] Han Cai, Junyan Li, Muyan Hu, Chuang Gan, and Song Han. EfficientViT: Multi-scale linear attention for high-resolution dense prediction. [vi](#)
- [2] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. [v](#)
- [3] Xiaohan Ding, Yuchen Guo, Guiguang Ding, and Jungong Han. ACNet: Strengthening the kernel skeletons for powerful CNN via asymmetric convolution blocks. [. iii](#)
- [4] Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding. Diverse branch block: Building a convolution as an inception-like unit. [. ii, iii, iv, v, vi, vii](#)
- [5] Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, and Xinchao Wang. Depgraph: Towards any structural pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16091–16101, 2023. [viii](#)
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. [i, v, vi, vii, viii](#)
- [7] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for MobileNetV3. [. vi, vii, viii](#)
- [8] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. [. vii](#)
- [9] Jie Hu, Li Shen, Samuel Albanie, Gang Sun, and Enhua Wu. Squeeze-and-excitation networks, 2019. [vi, vii, viii](#)
- [10] Mu Hu, Junyi Feng, Jiashen Hua, Baisheng Lai, Jianqiang Huang, Xiaojin Gong, and Xiansheng Hua. Online convolutional re-parameterization. [ii, iii, iv, vi, vii, viii](#)
- [11] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, 2009. [v](#)
- [12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems* 25, pages 1097–1105. Curran Associates, Inc., 2012. [vi, vii, viii](#)
- [13] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. [ii, vi](#)
- [14] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. [vi](#)
- [15] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. [vi](#)
- [16] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. [vii](#)
- [17] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. [vi, vii, viii](#)
- [18] Pavan Kumar Anasosalu Vasu, James Gabriel, Jeff Zhu, Oncel Tuzel, and Anurag Ranjan. Fastvit: A fast hybrid vision transformer using structural reparameterization. [vi](#)
- [19] Ao Wang, Hui Chen, Zijia Lin, Jungong Han, and Guiguang Ding. RepViT: Revisiting mobile CNN from ViT perspective. [i, ii](#)
- [20] Xiaohan Ding, Xiaohan Ding, Xiangyu Zhang, Xiangyu Zhang, Ningning Ma, Ningning Ma, Junwei Han, Jungong Han, Jungong Han, Guiguang Ding, Guiguang Ding, Jian Sun, Jian Sun, and Jian Sun. RepVGG: Making VGG-style ConvNets great again. pages 13733–13742. MAG ID: 3167976421. [ii, iii, vi](#)
- [21] Mingyang Zhang, Xinyi Yu, Jingtao Rong, and Linlin Ou. Repnas: Searching for efficient re-parameterizing blocks. In *2023 IEEE International Conference on Multimedia and Expo (ICME)*, pages 270–275. ISSN: 1945-788X. [ii](#)
- [22] Mingshu Zhao, Yi Luo, and Yong Ouyang. RepNeXt: A fast multi-scale CNN using structural reparameterization. [i, ii](#)